PMDM05.- Interacciones entre clases y actividades.

Caso práctico



Ron Lach (Pexels)

El equipo ha realizado los primeros diseños, mediante los cuales han adquirido muchos conocimientos.

- Una necesidad que me surge ahora -indica María-, es poder utilizar una nueva pantalla en la que se pueda solicitar al usuario alguna opción, fuera de la pantalla principal (activity_main) de la app.

- Bien visto -responde Ada-. Hasta ahora todas las apps desarrolladas han sido muy sencillas, sólo se han presentado diseños aislados de interfaces, es decir, no existen varias pantallas que interactúen entre sí o se pasen datos de unas a otras.

- ¡Claro! -exclama Juan-. Las aplicaciones constan de varias pantallas entrelazadas entre ellas.

- Ha llegado el momento de avanzar -explica Ada-. Vais a aprender a utilizar Intents, Splash Screen y los menús habituales de las aplicaciones de dispositivos para establecer la configuración de la aplicación, o para acceder a distintas opciones que no tiene sentido que se engloben en la actividad principal de la aplicación.

GOBIERNO DE ESPAÑA
MINISTERIO DE EDUCACIÓN Y FORMACIÓN PROFESIONAL

Ministerio de Educación y Formación Profesional. (Dominio público)

Materiales formativos de FP Online propiedad del Ministerio de Educación y Formación Profesional.

Aviso Legal

1.- Intents.

Caso práctico

Los Intents permiten que las pantallas interactúen entre sí, e incluso con otras aplicaciones.

- Os propongo un nuevo reto -explica Ada-. La app ha de ser capaz de abrir un archivo pdf.

- ¿Una pista? -solicita Juan.

- Para conseguirlo -prosigue Ada-, la app deberá encontrar una app en el dispositivo que sea capaz de abrir este tipo de archivos.

Esta y otras opciones de interacción se realizan gracias al objeto Intent, que podrá ser llamado de forma implícita o explícita.



Mikhail (Pexels)

Una **Intent** es un objeto de acción que usamos para solicitar una acción de otro componente de la aplicación. Aunque las intents facilitan la comunicación entre los componentes de muchas maneras, existen tres casos de uso fundamentales:

- **Para comenzar una actividad:**

  Una **Activity** representa una única pantalla en una aplicación. Puedes iniciar una nueva instancia de una **Activity** pasando una Intent a **startActivity()**. La **Intent** describe la actividad que se debe iniciar y contiene los datos necesarios para ello.

  Si deseas recibir un resultado de la actividad cuando finalice, llama a **startActivityForResult()**. La actividad recibe el resultado como un objeto **Intent** separado en el callback de **onActivityResult()** de la actividad.

- **Para iniciar un servicio:**

  Un **Service** es un componente que realiza operaciones en segundo plano sin una interfaz de usuario. Puede iniciar un servicio para realizar una operación única (como descargar un archivo) pasando una **Intent** a **startService()**. La **Intent** describe el servicio que se debe iniciar y contiene los datos necesarios para ello.

- **Para entregar un mensaje:**

  Un mensaje es un aviso que cualquier aplicación puede recibir. El sistema entrega varios mensajes de eventos del sistema, como cuando el sistema arranca o el dispositivo comienza a cargarse. Puedes enviar un mensaje a otras apps pasando una **Intent** a **sendBroadcast()**, **sendOrderedBroadcast()** o **sendStickyBroadcast()**.

1.1.- Tipos de Intents.

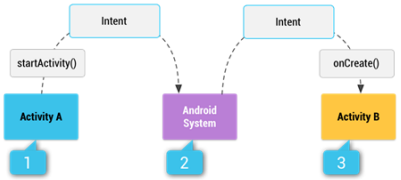
Existen dos tipos de intents:

- **Intents explícitas**: especifican qué componente se debe iniciar mediante su nombre (el nombre de clase completamente calificado). Usualmente, el usuario usa una intent explícita para iniciar un componente en su propia aplicación porque conoce el nombre de clase de la actividad o el servicio que desea iniciar. Por ejemplo, puede utilizarla para iniciar una actividad nueva en respuesta a una acción del usuario o iniciar un servicio para descargar un archivo en segundo plano.

  Cuando crea una **intent explícita** para iniciar una actividad o un servicio, la aplicación inicia inmediatamente el componente de la aplicación especificado en el objeto Intent.

- **Intents implícitas**: no se nombra el componente específicamente; pero, en cambio, se declara una acción general para realizar, lo que permite que un componente de otra aplicación la maneje. Por ejemplo, si desea mostrar al usuario una ubicación en un mapa, puede usar una intent implícita para solicitar que otra aplicación capaz muestre una ubicación específica en un mapa.

  Cuando crea una **intent implícita**, el sistema Android busca el componente apropiado para iniciar comparando el contenido de la intent con los ***filtros de intents*** declarados en el archivo de manifiesto (**AndroidManifest.xml**) de otras aplicaciones en el dispositivo. Si la intent coincide con un filtro de intents, el sistema inicia ese componente y le entrega el objeto **Intent**. Si varios filtros de intents son compatibles, el sistema muestra un cuadro de diálogo para que el usuario pueda elegir la aplicación que se debe usar.



Google Developers (Uso educativo nc)

Figura 1: Ilustración de la forma en que se entrega una intent implícita mediante el sistema para iniciar otra actividad. **[1]** La *actividad A* crea una **Intent** con una descripción de acción y la pasa a **startActivity()**. **[2]** El sistema Android busca en todas las apps un filtro de intents que coincida con la intent. Cuando se encuentra una coincidencia, **[3]** el sistema inicia la actividad coincidente (*actividad B*) invocando su método **onCreate()** y pasándolo a la **Intent**. Fuente: Android Developer.

Un **filtro de intents** es una expresión en el archivo de manifiesto de una aplicación que especifica el tipo de intents que el componente podría recibir. Por ejemplo, declarando un filtro de intents para una actividad, permite que otras aplicaciones inicien directamente la actividad con cierto tipo de intent. Así mismo, si *no* declara ningún filtro de intent para una actividad, solo se la puede iniciar con una intent explícita.

1.2.- Creación de una intent.

Un objeto **Intent** tiene información que el sistema Android usa para determinar qué componente debe iniciar (como el nombre exacto del componente o la categoría del componente que debe recibir la intent), además de información que el componente receptor usa para realizar la acción correctamente (por ejemplo, la acción que debe efectuar y los datos en los que debe actuar).

La información principal que contiene una **Intent** es la siguiente:

Nombre del componente, acción, datos, categoría, extras e indicadores.

De toda esa posible información, tendrá especial interés el paso de extras, que consiste en pares de valores clave que tienen información adicional necesaria para lograr la acción solicitada. Al igual que algunas acciones usan tipos particulares de URI de los datos, algunas acciones también usan extras particulares.

Podemos agregar datos de campos extra con varios métodos **putExtra()**, cada uno de los cuales acepta dos parámetros: el nombre clave y el valor. También puedes crear un objeto **Bundle** con todos los datos de campos extra y luego insertar el **Bundle** en el Intent con **putExtras()**.

1.3.- Ejemplo de una intent explícita.

Una intent explícita es una intent que se usa para iniciar un componente específico de la aplicación, como una actividad o un servicio particular en la aplicación. Para crear una intent explícita, define el nombre de componente para el objeto **Intent**, todas las otras propiedades de la intent son opcionales.

Ejemplo

Si creaste un servicio en tu app denominado **DownloadService**, diseñado para descargar un archivo de la Web, puedes iniciarlo con el siguiente código:

```
// Executed in an Activity, so 'this' is the Context

// The fileUrl is a string URL, such as "http://www.example.com/image.png"

Intent downloadIntent = new Intent(this, DownloadService.class);

downloadIntent.setData(Uri.parse(fileUrl));

startService(downloadIntent);
```

El constructor **Intent(Context, Class)** proporciona **Context** a la aplicación; y el componente, un objeto **Class**. De esta manera, la intent inicia explícitamente la clase **DownloadService** en la app.

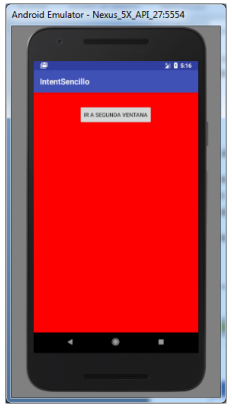
1.4.- Ejercicio resuelto 1.

Ejercicio Resuelto

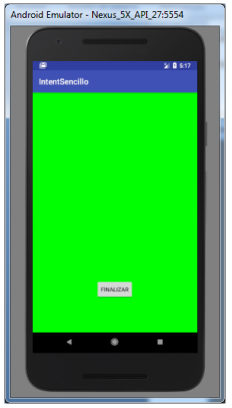
Crear una aplicación con 2 ventanas. la primera tendrá un fondo rojo y un botón que al ser clicado me lleve a la segunda ventana. La segunda ventana tendrá fondo verde y un botón que al ser clicado, hará que finalice la aplicación.

Mostrar retroalimentación

Tendremos que crear 2 actividades y vincularlas entre sí.



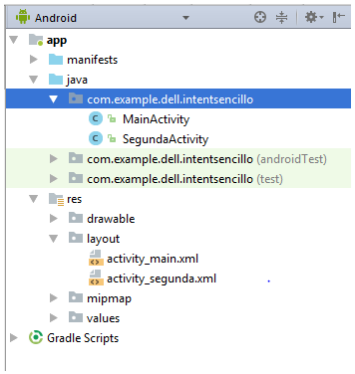
Google Developers (Uso educativo nc)



Google Developers (Uso educativo nc)

Para crear una nueva Activity **File->New->Activity->Empty Activity**

Quedará tal y como se muestra en la captura de Android.



Google Developers (Uso educativo nc)

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>

<android.support.constraint.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="#FF0000"
    tools:context="com.example.dell.intentsencillo.MainActivity">

    <Button
        android:id="@+id/btnIrSegundaActividad"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="32dp"
        android:text="Ir a segunda ventana"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintLeft_toLeftOf="parent"
        app:layout_constraintRight_toRightOf="parent"
```

```
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />
</android.support.constraint.ConstraintLayout>
```

activity_segunda.xml

```
<?xml version="1.0" encoding="utf-8"?>

<android.support.constraint.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="#00FF00"
    tools:context="com.example.dell.intentsencillo.SegundaActivity">
    <Button
        android:id="@+id/btnFinalizar"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginBottom="84dp"
        android:text="Finalizar"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintLeft_toLeftOf="parent"
        app:layout_constraintRight_toRightOf="parent" />
</android.support.constraint.ConstraintLayout>
```

MainActivity.java

```java
package com.example.dell.intentsencillo;
import android.content.Intent;
import android.support.v7.app.AppCompatActivity;
import android.os.Bundle;
import android.view.View;
import android.widget.Button;

public class MainActivity extends AppCompatActivity implements View.OnClickListener {
    Button btnIrSegundaActividad;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        btnIrSegundaActividad=(Button)findViewById(R.id.btnIrSegundaActividad);
        btnIrSegundaActividad.setOnClickListener(this);
    }

    @Override

    public void onClick(View v) {
        Intent i=new Intent(getApplicationContext(), SegundaActivity.class);
        startActivity(i);
    }
}
```

SegundaActivity.java

```java
package com.example.dell.intentsencillo;

import android.os.Build;
import android.support.v7.app.AppCompatActivity;
import android.os.Bundle;
import android.view.View;
import android.widget.Button;

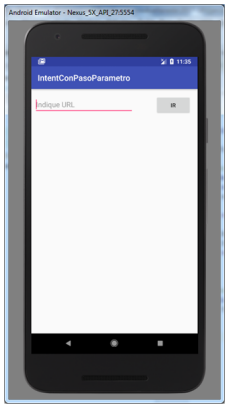
public class SegundaActivity extends AppCompatActivity {
    Button btnFinalizar;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_segunda);
        btnFinalizar=(Button)findViewById(R.id.btnFinalizar);
        btnFinalizar.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                //Para poder salir completamente de la aplicación no vale finish() ya que dicho método cerrar
                //Utilizaremos finishAffinity(), pero esta solo es válida a partir de la API 16
                if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.JELLY_BEAN) {
                    finishAffinity();
                }
            }
        });
    }
}

```

1.5.- Ejercicio resuelto 2.

Ejercicio Resuelto

Crea una aplicación que tenga un **intent** que cargue en una segunda actividad una página web a través de una **WebView**, de forma que se especifique la URL a cargar en la primera actividad.



Google Developers (Uso educativo nc)

Mostrar retroalimentación

activity_main.xml

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <android.support.constraint.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res/android"
```

```
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context="com.example.dell.intentconpasoparametro.MainActivity">

    <EditText
        android:id="@+id/etURL"
        android:layout_width="245dp"
        android:layout_height="wrap_content"
        android:layout_marginStart="8dp"
        android:layout_marginTop="16dp"
        android:hint="Indique URL"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent"
        tools:ignore="RtlCompat" />

    <Button
        android:id="@+id/btnIr"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginRight="16dp"
        android:layout_marginTop="16dp"
        android:text="Ir"
        app:layout_constraintRight_toRightOf="parent"
        app:layout_constraintTop_toTopOf="parent" />

</android.support.constraint.ConstraintLayout>
```

activity_segunda.xml

```xml
<?xml version="1.0" encoding="utf-8"?>
<android.support.constraint.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context="com.example.dell.intentconpasoparametro.SegundaActivity">

    <WebView
        android:id="@+id/wv"
        android:layout_width="match_parent"
        android:layout_height="400dp"
        android:layout_marginBottom="60dp"
        android:layout_marginTop="24dp"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintHorizontal_bias="0.0"
        app:layout_constraintLeft_toLeftOf="parent"
        app:layout_constraintRight_toRightOf="parent"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintVertical_bias="0.877"></WebView>

</android.support.constraint.ConstraintLayout>
```

mainActivity.java

```java
package com.example.dell.intentconpasoparametro;

import android.content.Intent;
```

```
import android.support.v7.app.AppCompatActivity;
import android.os.Bundle;
import android.view.View;
import android.widget.Button;
import android.widget.EditText;

public class MainActivity extends AppCompatActivity {
    EditText etURL;
    Button btnIr;

    @Override

    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        etURL=(EditText)findViewById(R.id.etURL);
        btnIr=(Button)findViewById(R.id.btnIr);
        btnIr.setOnClickListener(new View.OnClickListener() {

            @Override

            public void onClick(View v) {
                Intent i=new Intent(getApplicationContext(), SegundaActivity.class);
                i.putExtra("direccion", etURL.getText().toString());
                startActivity(i);
            }
        });

    }

}
```

SegundaActivity.java

```java
package com.example.dell.intentconpasoparametro;

import android.support.v7.app.AppCompatActivity;
import android.os.Bundle;
import android.view.View;
import android.webkit.WebView;
import android.widget.Button;

public class SegundaActivity extends AppCompatActivity {
    WebView wv;

    @Override

    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_segunda);
        wv=(WebView)findViewById(R.id.wv);
        Bundle bundle=getIntent().getExtras();
        String url=bundle.getString("direccion");
        wv.loadUrl("http://" + url);
    }
}
```

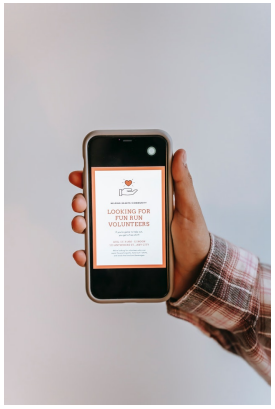
AndroidManifest.xml

Además, debemos incorporar en **AndroidManifest.xml** permiso de acceso a Internet.

```xml
<uses-permission android:name="android.permission.INTERNET"/>
```

2.- Splash screen.

Caso práctico



Liza Summer (Pexels)

- Enhorabuena -felicita Ada al equipo-. Habéis conseguido resolver el primer reto. Ahora os propongo un segundo reto.

- ¡Genial! -exclama María.

- Debéis realizar una app -explica Ada-, que muestre a los usuarios un mensaje cada vez que sea lanzada.

- ¿Cuál será el elemento nuevo a utilizar? -pregunta Pedro.

- En esta ocasión es el tipo de ventana denominada **Splash Screen** -responde Ada-, que permite a las apps mostrar información relevante cuando estas son lanzadas.

Es habitual que muchas aplicaciones incorporen una pantalla de presentación al iniciarse la aplicación, y que se muestra durante apenas unas decimas de segundo, o bien pocos segundos. A dicha pantalla se le suele llamar **Splash Screen**, y el objetivo es mejorar la experiencia de usuario, a la vez que proporcionar información adicional de esponsorización, patrocinio o información acerca de la empresa/s desarrolladora/s o colaboradora/s o bien, simplemente información comercial o publicitaria.

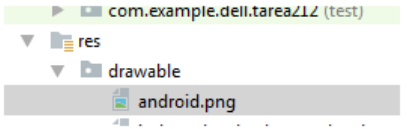
2.1.- Ejercicio Resuelto 3.

Ejercicio Resuelto

Construye una app Android que muestre una pantalla cuando se lance con una imagen que ocupe toda la pantalla del dispositivo. Al cerrarse se mostrará la pantalla por defecto de la app.

Mostrar retroalimentación

Lo primero que haremos será preparar preparar la imagen que utilizaremos. Lo ideal, al trabajar con imágenes en Android, será proporcionar diferentes resoluciones para que puedan adaptarse a los diferentes tamaños de pantallas disponibles en los dispositivos Android que usemos. En nuestro caso la llamaremos "android.png".



Google Developers (Uso educativo nc)

Tras buscar y adaptar la imagen que queremos que se muestre, definiremos una nueva Actividad que nosotros llamaremos *SplashScreenActivity* y que llevará asociado un fichero **activity_splash_screen.xml**. Dentro de este fichero .XML, incorporaremos un control **ImageView**, ocupando todo el *layout* mediante el uso del atributo **android:scaleType** con valor **fitXY**.

Archivo de manifiesto

Un archivo ordinario de manifiesto podría mostrar este aspecto donde se declaran las dos actividades y se arranca la principal.

```xml
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.dell.tarea212">

    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/AppTheme">
        <activity android:name=".MainActivity">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>

        <activity android:name=".SplashScreenActivity"></activity>

    </application>
</manifest>
```

Podemos apreciar como la actividad que tiene la etiqueta hija relacionada con la carga es MainActivity:

```xml
<category:android:name="android.intent.category.LAUNCHER" />
```

Esto significa que por defecto cargará la actividad principal. Nosotros lo cambiaremos para que inicialmente se cargue la actividad del splash screen, moviendo esta línea a la actividad Splash Screen.

Archivo de manifiesto

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.dell.tarea212">

    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/AppTheme">
        <activity android:name=".MainActivity"></activity>

        <activity android:name=".SplashScreenActivity">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>

    </application>
</manifest
```

SplashScreenActivity.java

Código java del splash screen.

```java
package com.example.dell.tarea53;

import android.content.Intent;
import android.content.pm.ActivityInfo;
import android.support.v7.app.AppCompatActivity;
import android.os.Bundle;
import android.view.Window;
import java.util.Timer;
import java.util.TimerTask;


public class SplashScreenActivity extends AppCompatActivity {

    @Override

    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        // Establecemos orientación exclusiva de retrato
        setRequestedOrientation(ActivityInfo.SCREEN_ORIENTATION_PORTRAIT);
        // Ocultamos la barra de titulo
        getSupportActionBar().hide();
        setContentView(R.layout.activity_splash_screen);

        TimerTask task = new TimerTask() {

            @Override

            public void run() {
                // Arrancamos la siguiente actividad
                Intent mainIntent = new Intent().setClass(
                        SplashScreenActivity.this, MainActivity.class);
                startActivity(mainIntent);
                // Cerramos esta actividad para que el usuario no pueda volver a ella mediante botón de volve
                finish();
            }
        };


        // Simulamos un tiempo en el proceso de carga durante el cual mostramos el splash screen
```

```
        Timer timer = new Timer();
        timer.schedule(task, 2000); //Tiempo de espera del temporizador en milisegundos
    }
}
```

En cuanto al **MainActivity.java** y el **activity_main.xml**, no haremos nada en particular, dejando el cásico *Hola Mundo!*. Por ello, tras los dos segundos de retardo, aparecerá la pantalla con el texto Hola Mundo!.



Google Developers (Uso educativo nc)



Google Developers (Uso educativo nc)

SplashScreenActivity.java

Una forma alternativa de programar el **SplashScreenActivity.java**, ayudándonos de los asistentes.

```java
package com.example.dell.splashscreen;

import android.content.Intent;
import android.os.Handler;
import android.support.v7.app.AppCompatActivity;
import android.os.Bundle;

public class SplashScreenActivity extends AppCompatActivity {

    @Override

    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_splash_screen);

        new Handler().postDelayed(new Runnable() {

            @Override

            public void run() {
                //Lanzaremos despues de 3000ms(3s), el MainActivity
                Intent intent=new Intent(SplashScreenActivity.this, MainActivity.class);
                startActivity(intent);
            }
        }, 3000);
    }
}
```

3.- ListView.

Caso práctico



Google Developers (Uso educativo nc)

- Vamos a por el tercer reto -informa Ada.

- ¿De qué se trata esta vez? -pregunta María.

- Ahora debéis incorporar listas a la aplicación -explica Ada-. Para ello utilizaréis el elemento **ListView**.

- ¡Es verdad! -exclama María-. Las listas se usan en las pantallas que muestran los contactos de una agenda, los mensajes de un chat, etc. La longitud y el número de elementos a mostrar es algo dinámico y cambia en cada instante dependiendo de la situación concreta.

- En vuestro caso -sigue explicando Ada-, las aplicaciones deben mostrar una lista de elementos conseguidos a partir de una consulta a distintos archivos que se encuentran en el dispositivo, como imágenes o documentos PDFs. Dependiendo del número de elementos disponibles en el dispositivo se mostrará un número de elementos u otros.

Un **ListView** es un grupo de vistas que muestra una lista de elementos desplazables. Los elementos de la lista se insertan automáticamente en la lista con un adaptador (clase **Adapter**) que toma contenido de una fuente, como una matriz o consulta de base de datos, y convierte cada resultado en una vista que se dispone en la lista.

Nos encontramos varias posibilidades, dependiendo de la complejidad que queramos atribuir a nuestro **ListView**. Concretamente, podremos usar un **ListView** con un **adaptador de tipo String** (es el caso más simple), o bien usar un **ListView** con un **adaptador personalizado** (caso más complejo y que trataremos en un apartado posterior).

Para insertar vistas de forma dinámica con un adaptador nos ayudaremos de la clase **AdapterView**. Y dentro de esta (extienden de ella), podremos usar la clase **ListView**, o también la clase **GridView**, de forma que enlazaremos la instancia **AdapterView** con un **Adapter**, que recupera datos de una fuente externa y crea una **View** que representa cada entrada de datos.

Android proporciona varias subclases de **Adapter** que resultan útiles para recuperar diferentes tipos de datos y generar vistas para una **AdapterView**. Los dos adaptadores más comunes son los siguientes:

- ArrayAdapter
- SimpleCursorAdapter

En particular nos centraremos en **ArrayAdapter**.

3.1.- ArrayAdapter.

Usaremos este adaptador cuando la fuente de datos sea una matriz. Según la configuración predeterminada, **ArrayAdapter** crearemos una vista para cada elemento de la matriz llamando a **toString()** en cada elemento y disponiendo los contenidos en una **TextView**.

Por ejemplo, si hay una matriz de strings que deseamos visualizar en una **ListView**, inicializaremos un nuevo **ArrayAdapter** usando un constructor para especificar el diseño de cada string y la matriz de strings:

```
1 | ArrayAdapter<String> adapter = new ArrayAdapter<String>(this, android.R.layout.simple_list_item_1, myStringArray);
```

Los argumentos para este constructor son los siguientes:

- El **Context** de tu app.
- El diseño que contiene un **TextView** para cada string de la matriz.
- La matriz de strings.

Luego, simplemente llamaremos a **setAdapter()** en nuestro **ListView**:

```
1 | ListView listView = (ListView) findViewById(R.id.listview);
2 | listView.setAdapter(adapter);
```

Para personalizar el aspecto de cada elemento, podemos anular el método **toString()** de los objetos de nuestra matriz. Como alternativa, si deseamos crear una vista para cada elemento que no sea una **TextView** (por ejemplo, si deseamos una **ImageView** para cada elemento de la matriz), extenderemos la clase de **ArrayAdapter** y anularemos **getView()** a fin de mostrar el tipo de vista que queramos para cada elemento.

Si durante el ciclo de vida de nuestra aplicación cambiamos los datos subyacentes que lee nuestro adaptador, debemos llamar a **notifyDataSetChanged()**. Esto le notificará a la vista anexada que se modificaron los datos y que debe actualizarse.

3.2.- Manejo de eventos de clicado.

Podemos atender a eventos de clicado en cada elemento de una **AdapterView** al implementar la interfaz **AdapterView.OnItemClickListener**.

Ejemplo

```
// Create a message handling object as an anonymous class.

private OnItemClickListener mMessageClickedHandler = new OnItemClickListener() {

    public void onItemClick(AdapterView parent, View v, int position, long id) {
        // Do something in response to the click
    }

};

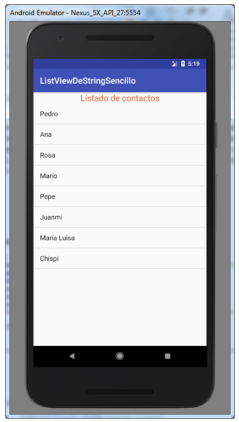
listView.setOnItemClickListener(mMessageClickedHandler);
```

3.3.- Ejemplo de ListView con adaptador de tipo String. Ejercicio resuelto 4.

Ejercicio Resuelto

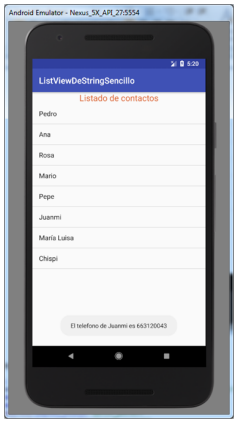
En este caso, vamos a tratar de crear un ListView sencillo, para lo cual trataremos de que lo que se muestre en cada ítem, sea simplemente una cadena de texto. Además dotaremos a cada ítem del ListView de la capacidad de responder al evento de clicado. Concretamente, nuestra aplicación será una sencilla agenda de contactos en la cual, al clicar en un contacto, se lanzará un Toast mostrando el número de teléfono asociado. El aspecto será el siguiente:

Pantalla inicial:



Google Developers (Uso educativo nc)

Tras clicar en Juanmi:



Google Developers (Uso educativo nc)

Mostrar retroalimentación

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>

<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    tools:context="com.example.dell.listviewdestringsencillo.MainActivity">

    <TextView
        android:id="@+id/tv1"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Listado de contactos"
        android:textSize="20dp"
        android:textColor="#DD6633"
        android:gravity="center"/>

    <ListView
        android:id="@+id/lv1"
        android:layout_width="match_parent"
        android:layout_height="match_parent"></ListView>

</LinearLayout>
```

MainActivity.java

```java
package com.example.dell.listviewdestringsencillo;

import android.app.AlertDialog;
import android.app.Dialog;
import android.content.DialogInterface;
import android.support.v7.app.AppCompatActivity;
import android.os.Bundle;
import android.view.View;
import android.widget.AdapterView;
import android.widget.ArrayAdapter;
import android.widget.ListView;
import android.widget.TextView;
import android.widget.Toast;

public class MainActivity extends AppCompatActivity {
    private String[] nombres = {"Pedro", "Ana", "Rosa", "Mario", "Pepe", "Juanmi", "María Luisa", "Chispi"};
    private String[] telefonos = { "658231234", "623123400", "912334489", "918732312", "657762598", "66312004
    private ListView lv1;
    private int posicion;

    @Override

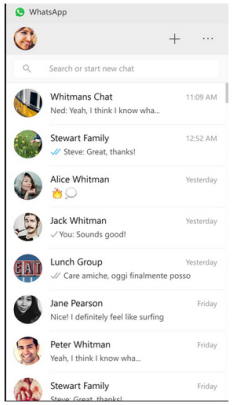
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        lv1 =(ListView)findViewById(R.id.lv1);
        ArrayAdapter<String> adapter = new ArrayAdapter<String>(this,android.R.layout.simple_list_item_1, nom
        lv1.setAdapter(adapter);
        lv1.setOnItemClickListener(new AdapterView.OnItemClickListener() {

            @Override

            public void onItemClick(AdapterView adapterView, View view, int i, long l) {
                posicion=i;
                Toast.makeText(MainActivity.this, "El telefono de " + nombres[i] + " es " + telefonos[i], Toa
            }
        });
    }
}
```

3.4.- ListView con adaptadores personalizados.

El caso más habitual de uso de un **ListView** es aquel en el cual los ítems que forman parte de dicho **ListView** sean un conjunto de componentes o vistas. Por ejemplo, en aplicaciones como la agenda telefónica de nuestro móvil, o como *Whatsapp*, se presentan unos listados (en realidad suelen estar desarrollados con una clase **RecyclerView**, que es una evolución de **ListView**), donde cada ítem, presenta varios componentes:



Google Developers (Uso educativo nc)

En la imagen anterior podemos ver que cada ítem del listado, a su vez está compuesto por una imagen, un nombre del chat, el último mensaje escrito y la fecha de ultima actualización del chat.

Esto pone de manifiesto que al final se hace necesario tratar de incorporar estructuras complejas que serán manejadas mejor si hacemos definiciones de clase que contengan todos estos componentes. Además, el ítem del **ListView** que queramos manejar, deberá ser capaz de escuchar eventos que sucedan sobre ellos, ya sean eventos sobre el ítem global (por ejemplo, para la imagen anterior, al clicar en el ítem, se abre una nueva **Activity** que me lleva al chat específico), o bien eventos sobre un componente del ítem (por ejemplo, para la imagen anterior, responder al clicado sobre la imagen para poder verla en grande en una nueva **Activity**).

Será necesario desarrollar un adaptador personalizado. Dicho adaptador se podrá crear dentro de la actividad principal, pero es más frecuente crearlo en un fichero específico aparte. Dicha clase extenderá de la clase **BaseAdapter** y dentro de ella definiremos un objeto de la clase **Context**, así como alguna estructura dinámica para contener cada uno de los ítems, normalmente un objeto **List** o **ArrayList**. Dicha estructura dinámica será una colección de objetos que se ajusten a nuestro universo. Por ejemplo, podremos crear un **ArrayList** de objetos de tipo **Contacto** para crear una agenda de teléfono.

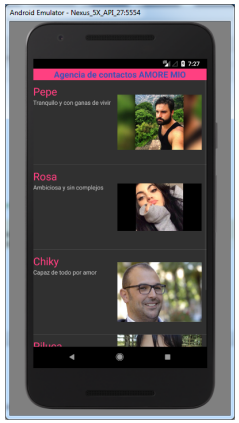
Al extender de la clase **BaseAdapter**, Android Studio nos forzará a sobrescribir los métodos **getCount()**, **getItem()**, **getItemId()** y **getView()**. De todos estos, sin duda el más importante será **getView()**, y con él configuraremos la apariencia o vista de cada ítem que tenga mi **ListView**. En el ejercicio resuelto siguiente se aclarará esto.

3.4.1.- Ejercicio resuelto 5.

Ejercicio Resuelto

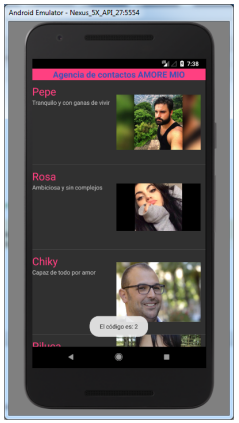
Vamos a crear una especie de agenda de contactos personales.

El aspecto que tendrá la aplicación será:



Google Developers (Uso educativo nc)

Al clicar en Rosa, me mostrará su código:



Google Developers (Uso educativo nc)

Mostrar retroalimentación

activity_main.xml

```xml
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical">

    <TextView
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Agencia de contactos AMORE MIO"
        android:background="@color/colorAccent"
        android:textColor="@color/colorPrimary"
        android:textSize="20dp"
        android:textStyle="bold"
        android:gravity="center"/>

    <ListView
        android:id="@+id/lv"
        android:layout_width="match_parent"
        android:layout_height="match_parent">
    </ListView>

</LinearLayout>
```

item.xml

Creamos un fichero XML adicional (**item.xml**) que contiene la distribución de cada ítem que aparecerá en el ListView

```xml
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="horizontal"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <LinearLayout
        android:layout_width="200dp"
        android:layout_height="200dp"
        android:orientation="vertical">

        <TextView
            android:id="@+id/tvNombre"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:textSize="25dp"
            android:textColor="@color/colorAccent"
            android:layout_marginTop="10dp"/>

        <TextView
            android:id="@+id/tvDescripcion"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginBottom="10dp"/>

    </LinearLayout>

    <ImageView
        android:id="@+id/imgFoto"
        android:layout_width="200dp"
        android:layout_height="200dp"
        android:src="@mipmap/ic_launcher"/>

</LinearLayout>
```

ModeloPersona

Creamos una clase java para caracterizar el comportamiento de un objeto de tipo persona a la que llamaremos "**ModeloPersona**":

```
1  package com.example.dell.proyecto006_listviewpersona;
2  public class ModeloPersona {
3      private String nombre;
4      private String descripcion;
5      private int codigo;
6      private int foto;
7
8      public String getNombre() {
9          return nombre;
10     }
11
12     public void setNombre(String nombre){
13         this.nombre=nombre;
14     }
15
16     public String getDescripcion() {
17         return descripcion;
18     }
19
20     public void setDescripcion(String descripcion) {
21         this.descripcion = descripcion;
22     }
23
24     public int getCodigo() {
25         return codigo;
26     }
27
28     public void setCodigo(int codigo) {
29         this.codigo = codigo;
30     }
```

```
31
32      public int getFoto() {
33          return foto;
34      }
35
36      public void setFoto(int foto) {
37          this.foto = foto;
38      }
39  }
```

MainActivity.java

```
1   package com.example.dell.proyecto006_listviewpersona;
2
3   import android.support.v7.app.AppCompatActivity;
4   import android.os.Bundle;
5   import android.util.Log;
6   import android.view.View;
7   import android.widget.AdapterView;
8   import android.widget.ListView;
9   import android.widget.Toast;
10  import java.util.ArrayList;
11
12  public class MainActivity extends AppCompatActivity {
13      private ListView lv;
14      private Adaptador adaptador;
15
16      @Override
17
18      protected void onCreate(Bundle savedInstanceState) {
19          super.onCreate(savedInstanceState);
20          setContentView(R.layout.activity_main);
```

```
21          ArrayList <ModeloPersona> personas = new ArrayList<ModeloPersona>();
22
23          ModeloPersona m1=new ModeloPersona();
24          m1.setNombre("Pepe");
25          m1.setDescripcion("Tranquilo y con ganas de vivir");
26          m1.setCodigo(1);
27          m1.setFoto(R.drawable.foto1);
28          ModeloPersona m2=new ModeloPersona();
29          m2.setNombre("Rosa");
30          m2.setDescripcion("Ambiciosa y sin complejos");
31          m2.setCodigo(2);
32          m2.setFoto(R.drawable.foto2);
33          ModeloPersona m3=new ModeloPersona();
34          m3.setNombre("Chiky");
35          m3.setDescripcion("Capaz de todo por amor");
36          m3.setCodigo(3);
37          m3.setFoto(R.drawable.foto3);
38          ModeloPersona m4=new ModeloPersona();
39          m4.setNombre("Piluca");
40          m4.setDescripcion("Herida pero dispuesta a perdonar");
41          m4.setCodigo(4);
42          m4.setFoto(R.drawable.foto4);
43
44          personas.add(m1);
45          personas.add(m2);
46          personas.add(m3);
47          personas.add(m4);
48          adaptador=new Adaptador(this, personas);
49
50          lv = (ListView)findViewById(R.id.lv);
51          lv.setAdapter(adaptador);
52
53          lv.setOnItemClickListener(new AdapterView.OnItemClickListener() {
54
55              @Override
56
57              public void onItemClick(AdapterView<?> adapterView, View view, int i, long l) {
58                  try{
59                      ModeloPersona persona=(ModeloPersona)adaptador.getItem(i);
```

```
                    Log.e("persona", persona.getCodigo()+"-"+persona.getNombre());
                    Toast.makeText(getBaseContext(), "El código es: "+persona.getCodigo(), Toast.LENGTH_LONG)
                } catch (Exception e){
                    e.printStackTrace();
                }
            }
        });
    }
}
```

Adaptador.java

Aquí va nuestra clase de tipo adaptador.

```
import java.util.ArrayList;

public class Adaptador extends BaseAdapter {
    private Context miContexto;
    private ArrayList <ModeloPersona> miArrayList;

    public Adaptador(Context miContexto, ArrayList<ModeloPersona> miArrayList){
        this.miContexto=miContexto;
        this.miArrayList=miArrayList;
    }

    @Override

    public int getCount() {
        return miArrayList.size();
    }

```

```
    @Override

    public Object getItem(int i) {
        return miArrayList.get(i);
    }

    @Override

    public long getItemId(int i) {
        return miArrayList.get(i).getCodigo();
    }

    @Override

    public View getView(int i, View view, ViewGroup viewGroup) {
        LayoutInflater layoutInflater=LayoutInflater.from(miContexto);
        view = layoutInflater.inflate(R.layout.item, null);
        TextView nombre=(TextView)view.findViewById(R.id.tvNombre);
        TextView descripcion=(TextView)view.findViewById(R.id.tvDescripcion);
        ImageView foto=(ImageView)view.findViewById(R.id.imgFoto);
        nombre.setText(miArrayList.get(i).getNombre());
        descripcion.setText((miArrayList.get(i).getDescripcion()));
        foto.setImageResource(miArrayList.get(i).getFoto());
        return view;
    }
}
```

4.- Menús.

Caso práctico



Mikhail (Pexels)

- Ahora -dice Juan- empieza a echar en falta los menús. Hacen que sea más fácil interactuar con los usuarios.

- Vamos a ello -anima Ada-. Normalmente nos encontramos con aplicaciones que usan tres tipos de menús: **menú de opciones** principales de la app o activity, normalmente accesible desde la propia barra de la aplicación; **menú contextual**, desplegado encima de un elemento cuando el usuario hace una pulsación larga sobre el mismo; **menú emergente**, desplegado en la aplicación en general.

Los menús serán componentes habituales en aplicaciones Android. Nos encontraremos estas tres posibles situaciones en relación al uso de menús:

- **Menú de opciones y barra de app.**

  El menú de opciones es la colección principal de elementos de menú para una actividad. Es donde debemos colocar las acciones que tienen un impacto global en la app, como “Buscar”, “Enviar un mensaje” y “Configuración”.

- **Menú contextual y modo de acción contextual.**

  Un menú contextual es un menú flotante que aparece cuando el usuario hace un clic largo en un elemento (mantiene pulsado un elemento durante aproximadamente un segundo). Proporciona acciones que afectan el contenido seleccionado o el marco contextual.
  En el menú de acción contextual se muestran los elementos de acción que afectan el contenido seleccionado en una barra en la parte superior de la pantalla y se permite al usuario seleccionar varios elementos.

- **Menú emergente.**

  Un menú emergente muestra una lista de elementos en una lista vertical que está anclada a la vista que invocó el menú. Es adecuado para proporcionar una ampliación de acciones relacionadas con contenido específico o para proporcionar opciones para una segunda parte de un comando. Las acciones en un menú emergente **no** deben afectar directamente el contenido correspondiente, para eso están las acciones contextuales. En cambio, el menú emergente es para acciones extendidas relacionadas con partes del contenido de la actividad.

4.1.- Definición de un menú en XML.

Para todos los tipos de menús, Android proporciona un formato XML estándar para definir los elementos del menú. En lugar de incorporar un menú en el código de la actividad, debes definir un menú y todos los elementos en un recurso de menú XML (se encontrará en **res/menu/fichero_menu.xml**). Luego, podremos utilizar dicho recurso de menú (cargarlo como un objeto **Menu**) en la actividad o el fragmento.

El uso del recurso de menú es una práctica recomendada por algunos motivos:

- Es más fácil visualizar la estructura del menú en XML.
- Separa el contenido del menú del código de comportamiento de la aplicación.
- Nos permite crear configuraciones alternativas del menú para diferentes versiones de la plataforma, tamaños de pantallas y otras configuraciones aprovechando el framework de recursos de la app.

Para definir el menú, crearemos un archivo XML dentro del directorio **res/menu/** del proyecto y desarrollaremos el menú con los siguientes elementos:

- **<menu>** Define un objeto **Menu**, que es un contenedor para elementos del menú. Un elemento <menu> debe ser el nodo raíz del archivo y puede tener uno o más elementos <item> y <group>.
- **<item>** Crea un **MenuItem**, que representa un único elemento en un menú. Este elemento puede contener un elemento <menu> anidado para crear un submenú.
- **<group>** Contenedor opcional e invisible para elementos **<item>**. Nos permite categorizar los elementos del menú para que compartan propiedades, como el estado de una actividad o visibilidad.

Ejemplo

Presentamos un menú de ejemplo denominado **game_menu.xml**:

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <menu xmlns:android="http://schemas.android.com/apk/res/android">
3     <item android:id="@+id/new_game"
```

```
        android:icon="@drawable/ic_new_game"
        android:title="@string/new_game"
        android:showAsAction="ifRoom"/>

    <item android:id="@+id/help"
        android:icon="@drawable/ic_help"
        android:title="@string/help" />

</menu>
```

El elemento **<item>** admite varios atributos que puedes usar para definir la apariencia y el comportamiento de un elemento. Los elementos del menú precedente incluyen estos atributos (estos son los más importantes, pero hay más):

- **android:id:** ID de un recurso que es exclusivo del elemento y permite que la aplicación lo reconozca cuando el usuario lo selecciona.
- **android:icon:** Referencia a un elemento de diseño para usar como el ícono del elemento.
- **android:title:** Referencia a una string para usar como el título del elemento.
- **android:showAsAction:** Especifica cuándo y cómo el elemento debe aparecer como un elemento de acción en la barra de app.

Podemos agregar un submenú a un elemento en cualquier menú (salvo en un submenú) agregando un elemento **<menu>** como campo secundario de un **<item>**. Los submenús son útiles cuando la aplicación tiene muchas funciones que se pueden organizar en temas, como elementos de una barra de menús de una aplicación para escritorio (Archivo, Editar, Ver, etc.).

Ejemplo

```
<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:android="http://schemas.android.com/apk/res/android">
    <item android:id="@+id/file"
        android:title="@string/file" >
        <!-- "file" submenu -->
        <menu>
```

```
            <item android:id="@+id/create_new"
                  android:title="@string/create_new" />
            <item android:id="@+id/open"
                  android:title="@string/open" />
        </menu>
    </item>
</menu>
```

Para usar un submenú en la actividad, tendremos que convertir el recurso XML en un objeto programable con **MenuInflater.inflate()**.

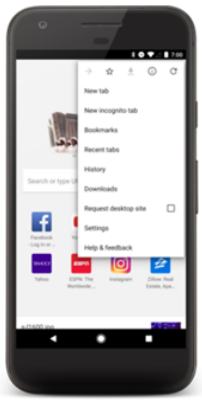
4.2.- Creación de menú de opciones.

El menú de opciones es dónde debes incluir las acciones y otras opciones que son relevantes para el contexto de la actividad actual, como "Buscar, "Redactar correo electrónico" y "Ajustes".

Que los elementos de las opciones aparezcan en el menú en la pantalla depende de la versión para la que desarrollemos la aplicación:

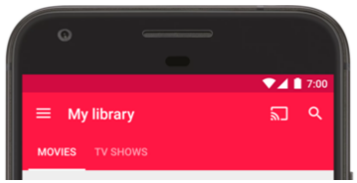
- Si desarrollamos la aplicación en **Android 2.3.x (nivel de API 10) o versiones anteriores**, el contenido del menú de opciones aparece en la parte inferior de la pantalla cuando el usuario presiona el botón *Menú*, como se muestra en la figura 1. Cuando se abre, la primera parte visible es el menú de íconos, que tiene hasta seis elementos de menú. Si el menú incluye más de seis elementos, Android coloca del sexto elemento en adelante en el menú ampliado, que se puede abrir seleccionando "**Más**".
- Si desarrollamos la aplicación en **Android 3.0 (nivel de API 11) y versiones posteriores**, los elementos del menú de opciones están disponibles en la barra de app. De forma predeterminada, el sistema dispone todos los elementos en acciones adicionales, que el usuario puede ver con el ícono de acciones adicionales a la derecha de la barra de app (o mediante el botón *Menú* del dispositivo si está disponible). Para permitir el acceso rápido a acciones importantes, podemos hacer que algunos elementos aparezcan en la barra de app agregando **android:showAsAction="ifRoom"** a los elementos **<item>** correspondientes (consulta la figura 2).

Figura 1



Google Developers (Uso educativo nc)

Figura 2



Google Developers (Uso educativo nc)

Podemos declarar elementos para el menú de opciones desde una **Activity** o desde un **Fragment**. Si tanto la actividad como los fragmentos declaran elementos para el menú de opciones, se los combina en la IU. Los elementos de la actividad aparecen primero y

después los elementos de cada fragmento, en el orden en que cada fragmento se agregó a la actividad. Si es necesario, puedes reordenar los elementos del menú con el atributo android:orderInCategory en cada **<item>** que necesites mover.

Para especificar el menú de opciones para una actividad, debemos sobrescribir el método **onCreateOptionsMenu()** (los fragmentos proporcionan su propio callback **onCreateOptionsMenu()**). En este método, podremos recoger el recurso de menú (definido en XML).

Ejemplo

```
@Override

public boolean onCreateOptionsMenu(Menu menu) {
    MenuInflater inflater = getMenuInflater();
    inflater.inflate(R.menu.game_menu, menu);
    return true;
}
```

También podemos agregar elementos del menú con **add()** y recuperar elementos con **findItem()**.

Si hemos desarrollado la aplicación para Android 2.3.x y versiones anteriores, el sistema llama a **onCreateOptionsMenu()** para crear el menú de opciones cuando el usuario abre el menú por primera vez. Si la desarrollaste para Android 3.0 y versiones posteriores, el sistema llama a **onCreateOptionsMenu()** cuando comienza la actividad, para mostrar los elementos de la barra de app.

4.3.- Manejo de eventos de clic.

Cuando el usuario selecciona un elemento del menú de opciones (incluidos los elementos de acción de la barra de app), el sistema llama al método **onOptionsItemSelected()** de la actividad. Este método pasa el **MenuItem** seleccionado. Podremos identificar el elemento si llamamos a **getItemId()**, que muestra el ID único del elemento del menú (definido por el atributo **android:id** en el recurso de menú o con un valor entero proporcionado al método **add()**). Podemos hacer coincidir este ID con elementos del menú conocidos para realizar la acción correspondiente.

Ejemplo

```
@Override

public boolean onOptionsItemSelected(MenuItem item) {
    // Handle item selection
    switch (item.getItemId()) {
        case R.id.new_game:
            newGame();
            return true;
        case R.id.help:
            showHelp();
            return true;
        default:
            return super.onOptionsItemSelected(item);
    }
}
```

Cuando controlemos correctamente un elemento del menú, mostraremos true. Si no controlas el elemento del menú, debes llamar a la implementación de la superclase de **onOptionsItemSelected()** (la implementación predeterminada muestra false).

Si la actividad incluye fragmentos, el sistema llama primero a **onOptionsItemSelected()** para la actividad y, a continuación, para cada fragmento (en el orden en que se agregaron) hasta que uno muestre true o se llamen todos los fragmentos.

Sugerencia: Desde Android 3.0 se incluye la capacidad de definir el comportamiento haciendo clic en un elemento del menú en XML, mediante el atributo android:onClick. El valor del atributo debe ser el nombre de un método definido por la actividad que usa el menú. El método debe ser público y aceptar un único parámetro **MenuItem**, cuando el sistema llama a este método, pasa el elemento del menú seleccionado.

Sugerencia: Si la aplicación contiene varias actividades y algunas de ellas proporcionan el mismo menú de opciones, debemos considerar crear una actividad que solo implemente los métodos **onCreateOptionsMenu()** y **onOptionsItemSelected()**. Luego, extenderemos esta clase por cada actividad que deba compartir el mismo menú de opciones. De esta manera, podemos administrar un conjunto de código para manejar las acciones del menú y cada clase descendiente hereda el comportamiento del menú. Si deseamos agregar elementos del menú a una de las actividades descendientes, invalidaremos sobrescribiendo **onCreateOptionsMenu()** en esa actividad. Llamaremos a **super.onCreateOptionsMenu(menu)** para que se creen los elementos originales del menú y luego agrega elementos nuevos con **menu.add()**. También puedes invalidar el comportamiento de la superclase para elementos del menú individuales.

4.4.- Creación de menús contextuales.



Google Developers (Uso educativo nc)

Un menú contextual ofrece acciones que afectan un elemento o marco contextual específicos en la IU. Puedes proporcionar un menú contextual para cualquier vista, aunque estos se usan con mayor frecuencia para elementos en **ListView**, **GridView** u otras colecciones de vistas en las cuales el usuario puede realizar acciones directamente en cada elemento.

En un menú contextual flotante. Un menú aparece como una lista flotante de elementos del menú (similar a un cuadro de diálogo) cuando el usuario realiza un clic largo (presiona y mantiene presionado) en una vista que declara admitir un menú contextual. Los usuarios pueden realizar una acción contextual en un elemento a la vez.

4.5.- Creación de un menú contextual flotante.

Para crear un menú contextual flotante:

1. Registraremos la **View** con la que se debe asociar el menú contextual llamando a **registerForContextMenu()** y pasándolo a la **View**.

Si la actividad usa un **ListView** o **GridView**, y deseamos que cada elemento proporcione el mismo menú contextual, registraremos todos los elementos de un menú contextual pasando **ListView** o **GridView** a **registerForContextMenu().**

2. Implementaremos el método **onCreateContextMenu()** en nuestra **Activity** o **Fragment**.

Cuando la vista registrada recibe un evento de clic largo, el sistema llama al método **onCreateContextMenu()**. Aquí es donde definimos los elementos del menú, por lo general agregando un recurso de menú.

Ejemplo

```
@Override

public void onCreateContextMenu(ContextMenu menu, View v, ContextMenuInfo menuInfo) {
    super.onCreateContextMenu(menu, v, menuInfo);
    MenuInflater inflater = getMenuInflater();
    inflater.inflate(R.menu.context_menu, menu);
}
```

MenuInflater permite agregar el menú contextual desde un recurso de menú. Los parámetros del método callback incluye la **View** que el usuario seleccionó y el objeto **ContextMenu.ContextMenuInfo** que proporciona información adicional sobre el elemento seleccionado. Si la actividad tiene varias vistas, cada unas de las cuales proporciona un menú contextual diferente, podremos usar estos parámetros para determinar qué menú contextual debemos agregar.

3. Implementaremos **onContextItemSelected()**.

Cuando un usuario selecciona un elemento del menú, el sistema llama a este método para que podamos realizar la acción apropiada.

Ejemplo

```
@Override

public boolean onContextItemSelected(MenuItem item) {
    AdapterContextMenuInfo info = (AdapterContextMenuInfo) item.getMenuInfo();
    switch (item.getItemId()) {
        case R.id.edit:
            editNote(info.id);
            return true;
        case R.id.delete:
            deleteNote(info.id);
            return true;
        default:
            return super.onContextItemSelected(item);
    }
}
```

El método **getItemId()** consulta el ID del elemento del menú seleccionado, que debemos asignar a cada elemento del menú en XML con el atributo **android:id**.

4.6.- Creación de un menú emergente.

Un **PopupMenu** es un menú modal anclado a una **View**. Aparece debajo de la vista anclada si hay espacio o, de lo contrario, sobre esta.

Si definimos el menú en XML, podemos hacer esto para mostrar el menú emergente:

1. Crearemos una instancia de **PopupMenu** con su constructor, que toma el **Context** de la aplicación actual y la **View** a la cual el menú debe anclarse.
2. Usaremos **MenuInflater** para inflar el recurso de menú hacia el objeto **Menu** devuelto por **PopupMenu.getMenu().**
3. Llamaremos a **PopupMenu.show()**

Ejemplo

Aquí se presenta un botón con el atributo **android:onClick** que muestra un menú emergente:

```
<ImageButton
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:src="@drawable/ic_overflow_holo_dark"
    android:contentDescription="@string/descr_overflow_button"
    android:onClick="showPopup" />
```

La actividad puede mostrar el menú emergente de la siguiente manera:

```
public void showPopup(View v) {
    PopupMenu popup = new PopupMenu(this, v);
    MenuInflater inflater = popup.getMenuInflater();
    inflater.inflate(R.menu.actions, popup.getMenu());

```

```
6       popup.show();
    }
```

En el nivel de API 14 y en niveles superiores, podemos combinar las dos líneas que agrandan el menú con **PopupMenu.inflate()**.

El menú se cierra cuando el usuario selecciona un elemento o toca fuera del área del menú. Puedes recibir el evento de cierre con **PopupMenu.OnDismissListener**.

4.7.- Manejo de eventos de clic.

Para realizar una acción cuando el usuario selecciona un elemento del menú, debes implementar la interfaz de **PopupMenu.OnMenuItemClickListener** y registrarla en el **PopupMenu** llamando a **setOnMenuItemclickListener().** Cuando el usuario selecciona un elemento, el sistema llama al callback **onMenuItemClick()** en la interfaz.

Ejemplo

```
public void showMenu(View v) {
    PopupMenu popup = new PopupMenu(this, v);
    // This activity implements OnMenuItemClickListener
    popup.setOnMenuItemClickListener(this);
    popup.inflate(R.menu.actions);
    popup.show();
}

@Override

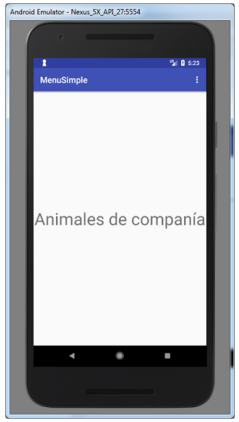
public boolean onMenuItemClick(MenuItem item) {
    switch (item.getItemId()) {
        case R.id.archive:
            archive(item);
            return true;
        case R.id.delete:
            delete(item);
            return true;
        default:
            return false;
    }
}
```

4.8.- Ejercicio resuelto 6: Items.

Ejercicio Resuelto

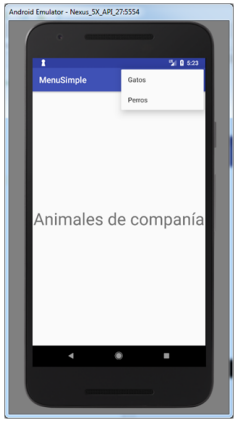
Vamos a tratar de crear un menú sobre la ActionBar de nuestra aplicación, de forma que aparezcan un par de opciones y que estas sean clicables. En nuestro caso simplemente abrirán una página web asociada al ítem clicado.

La apariencia inicial:



Google Developers (Uso educativo nc)

Al clicar en el menú, se desplegarán las opciones:



Google Developers (Uso educativo nc)

Al clicar en cada uno de los ítems de opción, se abrirá un página web:



Google Developers (Uso educativo nc)



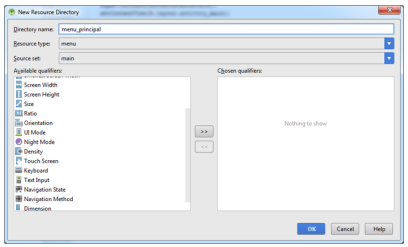
Google Developers (Uso educativo nc)

Mostrar retroalimentación

Crearemos un proyecto nuevo, y dentro de la carpeta de recursos **res**, crearemos un nuevo directorio de recursos que llamaremos **menu** (figura 1).

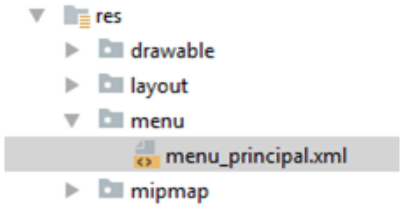
A su vez, dentro de dicha carpeta crearemos un fichero de recursos de Menú (**Menu Resources File**), que en mi caso llamaré **menu_principal.xml** (figura 2)**.**

Figura 1



Google Developers (Uso educativo nc)

Figura 2



Google Developers (Uso educativo nc)

menu_principal.xml

```
 <?xml version="1.0" encoding="utf-8" ?>
<menu xmlns:android="http://schemas.android.com/apk/res/android">
    <item
        android:id="@+id/itemGatos"
        android:title="Gatos" />
    <item
        android:id="@+id/itemPerros"
        android:title="Perros" />
</menu>
```

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<android.support.constraint.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context="com.example.dell.menusimple.MainActivity">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Animales de compañía"
```

```
        android:textSize="40dp"
        android:gravity="center"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintLeft_toLeftOf="parent"
        app:layout_constraintRight_toRightOf="parent"
        app:layout_constraintTop_toTopOf="parent" />

</android.support.constraint.ConstraintLayout>
```

MainActivity.java

```
package com.example.dell.menusimple;

import android.content.Intent;
import android.net.Uri;
import android.support.v7.app.AppCompatActivity;
import android.os.Bundle;
import android.view.Menu;
import android.view.MenuInflater;
import android.view.MenuItem;

public class MainActivity extends AppCompatActivity {

    @Override

    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }


    @Override
```

```
22
23      public boolean onCreateOptionsMenu(Menu menu) {
24          MenuInflater inflater=getMenuInflater();
25          inflater.inflate(R.menu.menu_principal, menu);
26          return true;
27      }
28
29      @Override
30
31      public boolean onOptionsItemSelected(MenuItem item) {
32          Intent i;
33          switch (item.getItemId()) {
34              case R.id.itemGatos: i = new Intent("android.intent.action.VIEW", Uri.parse("https://www.anipedia
35                  startActivity(i);
36                  break;
37              case R.id.itemPerros: i = new Intent("android.intent.action.VIEW", Uri.parse("https://www.anipedi
38                  startActivity(i);
39                  break;
40          }
41          return true;
42      }
43  }
```

AndroidManifest.xml

No debemos olvidar dar permisos de acceso a Internet en el **AndroidManifest.xml**:

```
1  <uses-permission android:name="android.permission.INTERNET"/>
```

4.9.- Ejercicio resuelto 7: SubItems.

Ejercicio Resuelto

Vamos a incorporar para el ejercicio anterior, en alguno de los ítems, subítems. Por ejemplo al pulsar el ítem Gatos, aparecerán sus ítems correspondientes.



Google Developers (Uso educativo nc)

Mostrar retroalimentación

menu_principal.xml

Copiamos el proyecto anterior y modificamos el fichero **menu_principal.xml** del menú de la siguiente forma:

```xml
<?xml version="1.0" encoding="utf-8" ?>
<menu xmlns:android="http://schemas.android.com/apk/res/android">
    <item
        android:id="@+id/itemGatos"
        android:title="Gatos" >
        <menu>
            <item
                android:id="@+id/itemGatos1"
                android:title="alimentacion"/>
            <item
                android:id="@+id/itemGatos2"
                android:title="cuidados en casa"/>
            <item
                android:id="@+id/itemGatos3"
                android:title="vacunas"/>
        </menu>
    </item>
    <item
        android:id="@+id/itemPerros"
        android:title="Perros" />
</menu>
```

MainActivity.java

...y por otro lado **MainActivity.java**:

```java
package com.example.dell.menusimplesubitems;

import android.content.Intent;
import android.net.Uri;
import android.support.v7.app.AppCompatActivity;
import android.os.Bundle;
```

```java
import android.view.Menu;
import android.view.MenuInflater;
import android.view.MenuItem;

public class MainActivity extends AppCompatActivity {

    @Override

    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }

    @Override
    public boolean onCreateOptionsMenu(Menu menu) {
        MenuInflater inflater=getMenuInflater();
        inflater.inflate(R.menu.menu_principal, menu);
        return true;
    }

    @Override

    public boolean onOptionsItemSelected(MenuItem item) {
        Intent i;
        switch (item.getItemId()) {
            case R.id.itemGatos1: i = new Intent("android.intent.action.VIEW", Uri.parse("https://es.wikipedi
                startActivity(i);
                break;
            case R.id.itemGatos2: i = new Intent("android.intent.action.VIEW", Uri.parse("https://www.tiendan
                startActivity(i);
                break;
            case R.id.itemGatos3: i = new Intent("android.intent.action.VIEW", Uri.parse("https://www.tiendan
                startActivity(i);
                break;
            case R.id.itemPerros: i = new Intent("android.intent.action.VIEW", Uri.parse("https://www.anipedi
                startActivity(i);
                break;
        }
        return true;
```

```
45          }
46      }
47
```

4.10.- Ejercicio resuelto 8: Menú contextual.

Ejercicio Resuelto

En este caso, vamos a hacer un ejercicio donde trabajemos con menús contextuales. Concretamente, mi menú contextual aparecerá al hacer clic sobre un **TextView**, aparecerá un menú emergente que permitirá cambiar el color de fondo de un layout interno.

La apariencia inicial, será:



Google Developers (Uso educativo nc)

Al mantener pulsado sobre el texto, se lanzará el menú emergente:



Google Developers (Uso educativo nc)

Si elijo, por ejemplo "Color rojo":

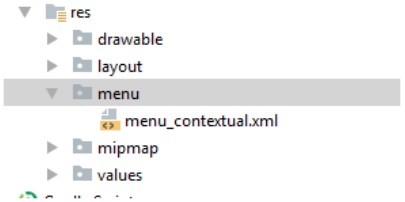


Google Developers (Uso educativo nc)

Mostrar retroalimentación

Crearemos un fichero **menu_contextual.xml**, dentro de la carpeta de recursos menu.

Su contenido será **menu_contextual.xml.**



Google Developers (Uso educativo nc)

menu_contextual.xml

```xml
<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:android="http://schemas.android.com/apk/res/android">
    <item
        android:id="@+id/itemRojo"
        android:title="Color rojo" />
    <item
        android:id="@+id/itemVerde"
        android:title="Color verde" />
    <item
        android:id="@+id/itemAzul"
        android:title="Color azul" />
</menu>
```

activity_main.xml

```xml
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    tools:context="com.example.dell.menucontextualbasico.MainActivity">

    <TextView
        android:id="@+id/tv1"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Mantenme pulsado para cambiar color de fondo"
        android:gravity="center"
        android:textSize="25dp"
        android:background="#FFFFFF"/>

    <LinearLayout
        android:id="@+id/layoutInterno"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:orientation="vertical">
    </LinearLayout>

</LinearLayout>
```

MainActivity.java

```java
package com.example.dell.menucontextualbasico;
import android.graphics.Color;
import android.support.v7.app.AppCompatActivity;
import android.os.Bundle;
import android.text.Layout;
import android.view.ContextMenu;
import android.view.MenuInflater;
import android.view.MenuItem;
import android.view.View;
import android.widget.EditText;
import android.widget.LinearLayout;
import android.widget.TextView;

public class MainActivity extends AppCompatActivity {
    TextView tv1;
    LinearLayout layoutInterno;

    @Override

    public void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        layoutInterno=(LinearLayout)findViewById(R.id.layoutInterno);
        tv1=(TextView)findViewById(R.id.tv1);
        registerForContextMenu(tv1);
    }

    @Override

    public void onCreateContextMenu(ContextMenu menu, View v, ContextMenu.ContextMenuInfo menuInfo)
    {
        menu.setHeaderTitle("Elija el color de fondo:");
        MenuInflater inflater = getMenuInflater();
        inflater.inflate(R.menu.menu_contextual, menu);
    }

```

```
    @Override

    public boolean onContextItemSelected(MenuItem item) {
        switch (item.getItemId()) {
            case R.id.itemRojo:layoutInterno.setBackgroundColor(Color.rgb(255, 0, 0)) ;
                break;
            case R.id.itemVerde:layoutInterno.setBackgroundColor(Color.rgb(0, 255, 0)) ;
                break;
            case R.id.itemAzul:layoutInterno.setBackgroundColor(Color.rgb(0, 0, 255)) ;
                break;
        }
        return true;
    }
}
```